

Poly-time reductions

Q. Which problems will we ~~be able to~~ ^{can} solve in practice?

A. $\rightarrow$ Those with polynomial-time algorithms.

Shortest path

min cut

2-satisfiability

planar 4-colorability

bipartite vertex cover

matching

primality testing

linear programming

longest path

max cut

3-satisfiability

planar 3-colorability

vertex cover

3d-matching

factoring

Integer linear programming.

Objective: classify problems according to polynomial time solvable and non-solvable.

Probably requires exponential time:-

* Given a constant-size program, does it halt in at most k -steps?

* Chess-problem.

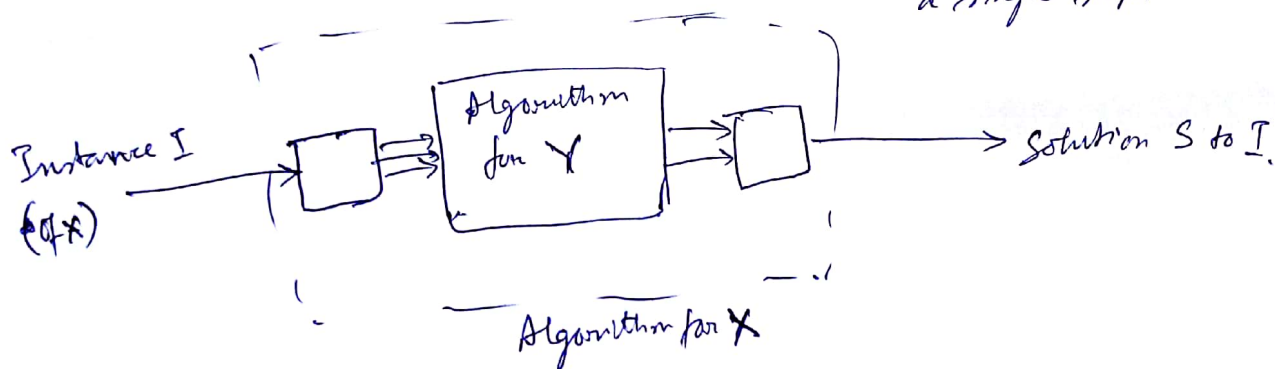
$\Rightarrow$ Huge number of problem, we ~~can~~ ^{could} not classify in that manner.

Polynomial-time Reduction:-

Problem X polynomial-time (cook) reduces to problem Y if arbitrary instances of problem X can be solved using:

- * Polynomial number of standard computational steps, plus
- * Polynomial number of calls to oracle that solves problem Y .

↑
A special piece of hardware that solves instances of Y in a single step.



Notation! $X \leq_p Y$.

Purpose:

1. Design algorithms: If $X \leq_p Y$ and Y can be solved in polynomial time, then X can be solved in polynomial time.
2. Establish intractability: If $X \leq_p Y$ and X cannot be solved in polynomial time, then Y cannot be solved in polynomial time.

CS 503 - Intractability

③

Establish equivalence: If both $X \leq_p Y$ and $Y \leq_p X$, we use notation $X \equiv_p Y$.

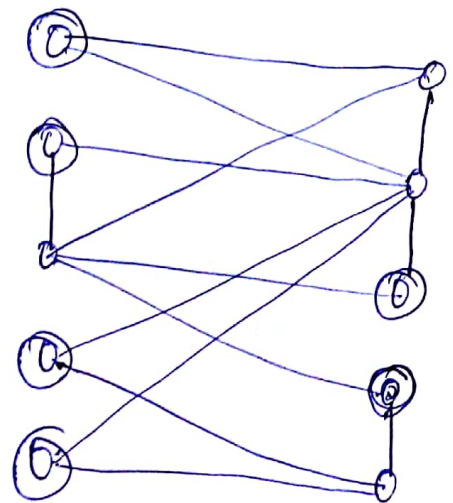
X can be solved in polynomial time iff Y can be.

⇒ Reductions classify problems according to relative difficulty.

Independent Set vs Vertex Cover

① Independent Set: Given a graph $G = (V, E)$ and an integer k , is there a subset of vertices $S \subseteq V$ such that $|S| \geq k$, and for each edge at most one of its endpoints is in S ?

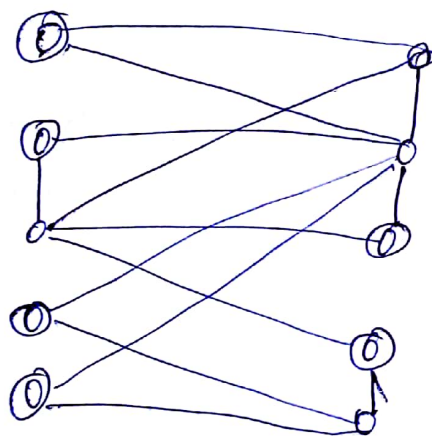
→ Is there an independent set of size ≥ 6 ?
size ≥ 7 ?



Independent set
of size 6.

Vertex-Cover: Given a graph $G=(V,E)$ and an integer k ,
is there a subset of vertices $S \subseteq V$ such that $|S| \leq k$,
and for each edge, at least one of its endpoints is in
 S ?

Is there a vertex cover of size ≤ 4 ?
size ≤ 3 ?



① independent set
of size 6

② vertex cover
of size 4.

Theorem: Vertex-Cover $\equiv$ Independent-Set.

Proof: We show S is an independent set of size k iff
 $V-S$ is a vertex cover of size $n-k$.

$\Rightarrow$. Let S be any I-set of size k .

• $V-S$ is of size $n-k$.

• Consider an arbitrary edge (u,v) .

• S independent $\Rightarrow$ either $u \notin S$ or $v \notin S$ (or both)

$\Rightarrow$ either $u \in V-S$ or $v \in V-S$ (or both).

• Thus $V-S$ covers (u,v) .

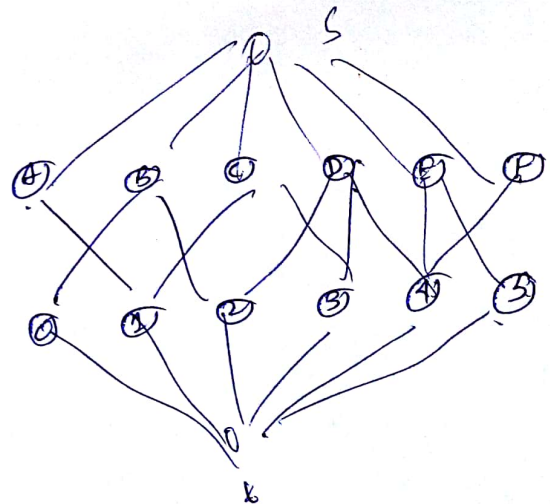
$V \setminus S$

- Let $V \setminus S$ be any vertex cover of size $n - k$
- S is of size k .
- Consider two nodes $u \in S$ and $v \in S$
- Observe that $(u, v) \notin E$ since $V \setminus S$ is a vertex cover
- Thus, no two nodes in S are joined by an edge $\Rightarrow S$ independent set. $\square$

④ Bipartite matching $\leq$ Max-flow

- $\rightarrow$ Construct new vertices s and t
- $\rightarrow$ add edges from s to each vertex in one set.
- $\rightarrow$ add edges from each vertex in other set to t .
- $\rightarrow$ Set all edge capacity to 1
- $\rightarrow$ Find maxflow in resulting network.
- $\rightarrow$ matching is the ~~edges~~ flow edges between two sets.

Cost of matching = Cost of max flow + Cost of reduction
linear.



img.

2

7 words
nab
km

CS 503- Intractability

Set-Cover: Given a set U of elements, a collection $S_1, S_2, \dots, S_n$ of subsets of U , and an integer k , does there exist a collection of $\leq k$ of these sets whose union is equal to U ?

Sample application:

- m available pieces of software.
- Set U of n capabilities that we would like our system to have.
- The i th piece of software provides the set $S_i \subseteq U$ of capabilities.
- Goal: achieve all n capabilities using fewest pieces of software.

$$U = \{1, 2, 3, 4, 5, 6, 7\}$$

$$S_1 = \{3, 7\}$$

$$S_4 = \{2, 4\}$$

$$S_2 = \{3, 4, 5, 6\}$$

$$S_5 = \{5\}$$

$$S_3 = \{1\}$$

$$S_6 = \{1, 2, 6, 7\}$$

$$k=2$$

Theorem: Vertex-cover $\leq_p$ Set-cover

Proof: Given a vertex-cover instance $G = (V, E)$, we construct a set-cover instance (U, S) that has a set cover of size k iff G has a vertex cover of size k .

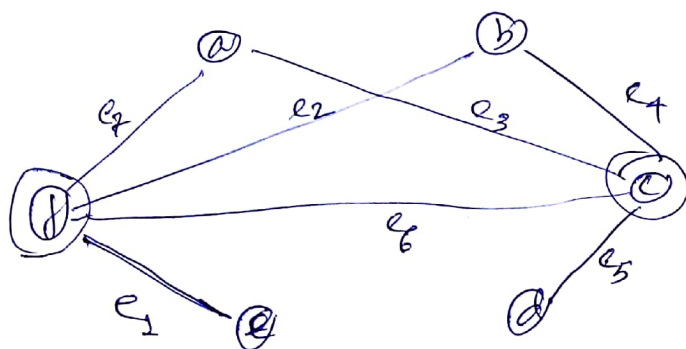
CS 503 - Intractability

8

Construction!

→ Universe $U = E$

→ Include one set for each node $v \in V$: $S_v = \{e \in E : e \text{ incident to } v\}$



$$U = \{1, 2, 3, 4, 5, 6, 7\}$$

$$S_a = \{3, 7\} \quad S_b = \{2, 4\}$$

$$S_c = \{3, 4, 5, 6\} \quad S_d = \{5\}$$

$$S_e = \{1\} \quad S_f = \{1, 2, 5, 7\}$$

Vertex cover instance
 $k=2$

Set cover instance
 $k=2$

Lemma: $G = (V, E)$ contains a vertex cover of size k iff (U, S) contains a set cover of size k .

Proof: $\Rightarrow$ Let $X \subseteq V$ be a vertex cover of size k in G .

$\Rightarrow$ Then $Y = \{S_v : v \in X\}$ is a set cover of size k .

$\Leftarrow$ Let $Y \subseteq S$ be a set cover of size k in (U, S) .

$\Rightarrow$ Then $X = \{v : S_v \in Y\}$ is a vertex cover of size k in G . $\square$

CS 503 - Intractability :

Decision Problem:

- Problem X is a set of strings
- Instance s is one string
- Algorithm A solves problem X : $A(s) = \text{yes}$ iff $s \in X$.

Def: Algorithm A runs in polynomial time if for every string s , $A(s)$ terminates in at most $P(|s|)$ "steps", where $P(\cdot)$ is some polynomial.

Def: Algorithm $C(s, t)$ is a certifier for problem X if for every string s , $s \in X$ iff there exists a string t , such that $C(s, t) = \text{yes}$.
↑
certificate.

NP! - NP is the set of problems for which there exists a poly-time certifier.

- $C(s, t)$ is poly-time algorithm
- Certificate t is of polynomial size: $|t| \leq P(|s|)$

Example: (3-Sat) | Hamiltonian path

Given a CNF formula Φ , is there a satisfying assignment?

Certificate: An assignment of truth values to the n boolean variables.

Certifier: check that each clause in Φ has at least one true literal.

Instance s : $\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$.

Certificate t : $x_1 = 1, x_2 = 1, x_3 = 0, x_4 = 0$ | Conclusion 3-Sat $\in$ NP.

P, NP, EXP

P $\rightarrow$ Decision problems solved by poly-time algorithm

NP $\rightarrow$ " " which have poly-time certifier.

EXP $\rightarrow$ " " solved by exponential time algo.

Claim: $P \subseteq NP$

$NP \subseteq EXP$

Proof: By def. there exists a poly-time certifier $C(S, t)$ for x .

To solve input S , run $C(S, t)$ on all strings t with $|t| \leq P(|S|)$.

Returns yes if $C(S, t)$ returns yes for any of these potential certificates.

$\Rightarrow P \subseteq EXP$

$P \approx NP$

Is the decision problem as easy as the certification problem?

If yes! ~~Proof~~

Polynomial transformation

Problem X polynomial transforms to problem Y if given any input x to X , we can construct an input y ^{in poly-time} such that x is a yes instance of X iff y is a yes instance of Y .

$|y|$ must be of size polynomial in $|x|$.

CS 503 - Intractability

(11)

Polynomial reduction $\rightarrow$ polynomial number of call to ^{oracle of} Y .

Polynomial transformation $\rightarrow$ (a) Only one call to the oracle

Open Question: Are they same (b) with respect to NP?

NP-complete: A problem $Y \in NP$ with property that for every problem $X \in NP$, $X \leq_p Y$.

Theorem: Suppose $Y \in NP$ -complete. Then $Y \in P$ iff $P = NP$

Proof: $\Leftarrow$ If $P = NP$, then $Y \in P$ because $Y \in NP$

$\Rightarrow$ Suppose $Y \in P$

$\rightarrow$ Consider any problem $X \in NP$. ~~Since~~

$\Rightarrow X \in P$, since $X \leq_p Y$

$\rightarrow$ This implies $NP \subseteq P$

- We already know that $P \subseteq NP \Rightarrow P = NP$

First NP-complete problem

Circuit-Sat $\in NP$ -complete.

(a) Read yourself the proof

CS 503- Intractability

How to prove that $Y \in \text{NP-complete}$.

Step 1: Show that $Y \in \text{NP}$

Step 2: Choose an NP-complete problem X

Step 3: Prove the $X \leq_p Y$

Theorem: If $X \in \text{NP-complete}$, $Y \in \text{NP}$, and $X \leq_p Y$, then $Y \in \text{NPC}$.

Proof: Consider any problem $W \in \text{NP}$

$\Rightarrow$ both $W \leq_p X$ and $X \leq_p Y$
 $\uparrow$ definition $\uparrow$ assumption.

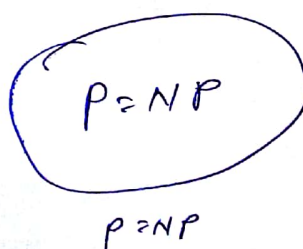
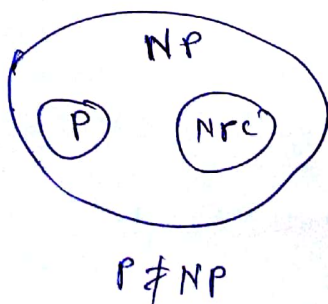
- by transitivity, $W \leq_p Y$.

$\Rightarrow Y \in \text{NPC}$.

$\Rightarrow$ Most NP problems are known to be either in P or NPC.

Exception: Factor, Graph-Isomorphism, ...

Theory: [Ladner 1975] Unless $P = \text{NP}$, there exist problems in NP that are neither in P nor in NPC.



CS 503 - Intractability

NP-Hard :- A problem such that every problem in NP polynomially time reduces to it.

~~Ex~~ an NP-H but not NPC $\rightarrow$ Halting problem

If we have to solve an NP-hard problem:

$\Rightarrow$ Sacrifice one of three desired features.

1. Solve arbitrary instances of the problem
2. Solve problem to optimality
3. Solve problem in polynomial time

Strategies:

- (i) Design algorithms for special cases of the problem.
- (ii) Design approximation algorithms or heuristics.
- (iii) Design algorithms that may take exponential time.

Example: Independent set on trees! Given a tree, find a maximum cardinality subset of nodes such that no two are adjacent.

Fact: A tree has at least one node that is a leaf (degree = 1)

Observation: If node v is a leaf, there exists a max cardinality independent set containing v .

Proof:

→ Consider a max cardinality independent set S .

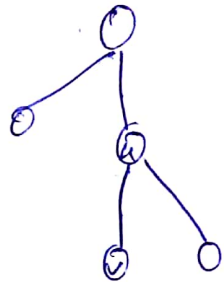
→ If $v \in S$, we are done

→ Let (u, v) be some edge.

- If $u \notin S$ and $v \notin S$, then $S \cup \{v\}$ is independent $\Rightarrow$
 S is not maximum.

- If $u \in S$ and $v \notin S$, then $S \cup \{v\} - \{u\}$ is independent.

□



Algo: Independent-Set-in-A-Forest (F)

$S \leftarrow \emptyset$

While (F has at least 1 edge)

$e \leftarrow (u, v)$ such that v is a leaf

$S \leftarrow S \cup \{v\}$

$F \leftarrow F - \{(u, v)\}$ & delete u and v and all incident edges.

Return $S \cup \{\text{nodes remaining in } F\}$

Time complexity: $O(n)$ by considering nodes in postorder.

Theorem: Above greedy algorithm finds a max cardinality independent set in forests (and hence trees).

Polynomial-time special cases NP-hard problems.

Trees!: Vertex-cover, Independent-Set, Graph-Isomorphism.

Bipartite graphs: Vertex-cover, 2-color

~~Chordal graphs:~~

~~Planar graph: Max-cut, etc~~

Approximation algorithm.

- Guaranteed to run in poly-time
- " " solve arbitrary instance of the problem.
- " " find solution within ratio ρ of true optimum.

Example! Given a graph G , the greedy algorithm finds a vertex cover that uses $\leq 2 \text{OPT}(G)$ vertices in $O(m+n)$ time.

$$\rho = 2$$

Challenge! Need to prove a solution's value is close to optimum value, without even knowing what optimum value is.

The cost/profit for approximation algo $\rightarrow c$

The cost/profit for optimal solution $\rightarrow c^*$

Approximation ratio $\rightarrow \rho(n) \geq \max\left(\frac{c}{c^*}, \frac{c^*}{c}\right)$.

The approximation algorithm is called $\rho(n)$ -approximation algorithm.